Advanced Web Programming

SIB245007 | D-IV Sistem Informasi Bisnis

Meeting 9: **Data Processing: Import, Export, and Queues**

From the ETL pipeline, large datasets, and memory, to heavy work handed off to the background

What You'll Learn

1. Explain the **ETL** pattern and **batch processing** for handling large amounts of data
2. Build a date-range report, and pick the right techniques for **large datasets**
3. Understand why **memory** runs out, and choose memory-friendly ways to read data
4. Choose between **CSV, Excel, and PDF** for a need
5. Follow a **job** through a queue, from creation to execution by a worker, and run it in **production**

> These slides cover the concepts. Applying reports, export/import, and queues to Simple POS is done in the lab jobsheet.

Meeting Map



> Like a restaurant kitchen: ingredients come in (**extract**), get cooked (**transform**), then are served (**load**). When orders pile up, guests get a table number and the food is brought over later. That is a queue.

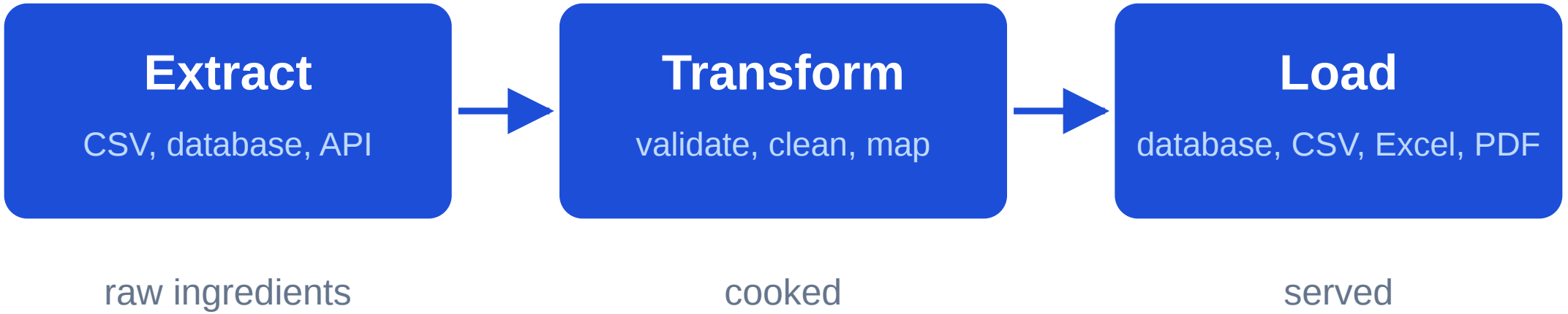
Part 1

ETL and Batch Processing

Four Basic Terms

Term	Meaning
ETL	Extract (take), Transform (reshape), Load (store or present)
Batch processing	processing data in groups, not one by one
Import / Export	data in from CSV/Excel to the database, or out to CSV/Excel/PDF
Queue	heavy work handed off, done by another process in the background

ETL: Three Stages That Always Appear



- Keep the three stages separate so each part is easy to test and replace
- Source, validation rules, and destination can change without touching the others

Import and Export Are Two-Way ETL

	Extract	Transform	Load
Import	read CSV / Excel file	validate each row, map columns	save to database
Export	query the database	pick columns, format dates and numbers	write CSV / Excel / PDF

> For every feature, ask: where does the data come from, what changes, where does the result go?

Batch: One by One vs One Truckload

```php
// 10,000 rows = 10,000 queries
foreach ($rows as $row) {
    Article::create($row);
}

// 10,000 rows = 20 queries (batches of 500 rows)
foreach (array_chunk($rows, 500) as $batch) {
    Article::insert($batch);
}
```

Every query pays a trip to the database. One full truck beats 10,000 motorbike couriers.

> Bulk `insert()` does not fill `created_at` / `updated_at` and does not fire model events. Fill timestamps yourself if needed.

Batch Size and Reading in Batches

Too small

- Too many round trips
- Slow

Too large

- Memory balloons
- One failure fails many rows

```php
Article::where('status', 'draft')
    ->chunkById(500, function ($articles) {
        foreach ($articles as $a) {
            // process each row
        }
    });
```

> Start with hundreds to a thousand rows per batch, then measure. Wrap each batch in `DB::transaction()` if it must succeed or fail together.

Idempotent: Safe to Run Many Times

Uploading the same import file twice must not create duplicates.

```
Article::updateOrCreate(
    ['slug' => $row['slug']],     // lookup key
    ['title' => $row['title']],   // values to update
);

// batch version: one query for many rows
Article::upsert($batch, ['slug'], ['title']);
```

`upsert()` needs a **unique index** on the key columns. Without it, the database cannot tell which rows count as the same.

Part 2

Reports and Large Datasets

List vs Report

- **List**: shows rows one by one
- **Report**: summarizes, for example "total X in a date range"

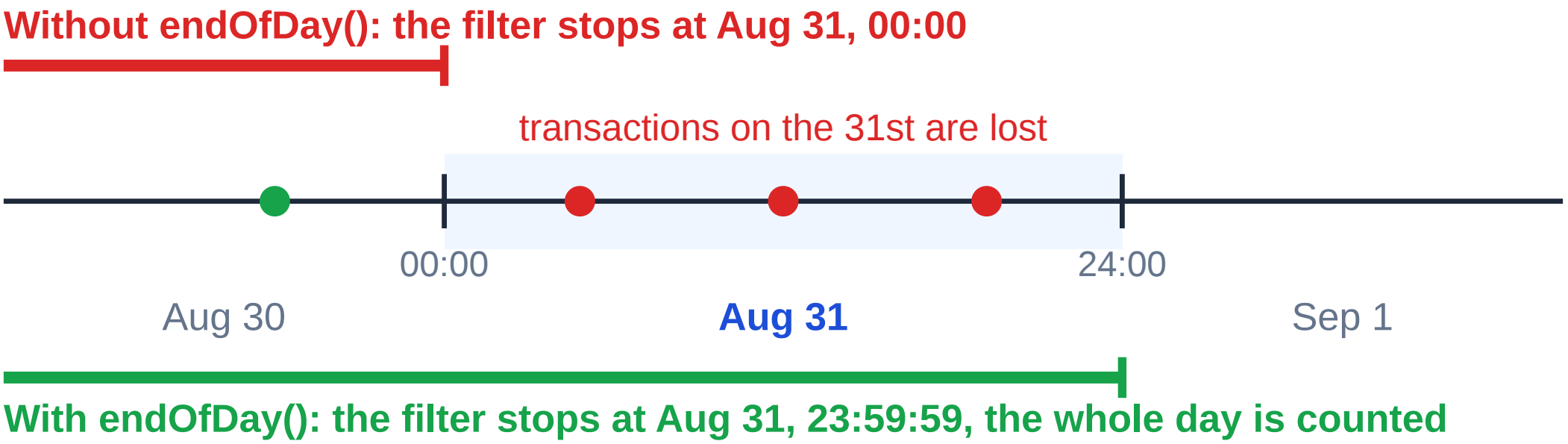
```
$totalWords = Article::whereBetween('published_at', [$from, $to])
    ->sum('word_count');   // filter by date, then sum one column
```

Common Trap: The Cut-Off Last Day

> A filter "1 to 31 August" means "up to 31 August 00:00:00". Every transaction on the 31st is lost.

```
$from = $request->date('from')->startOfDay();   // 00:00:00
$to = $request->date('to')->endOfDay();         // 23:59:59
```

The Day-Boundary Trap, Drawn on a Timeline



Compute in the Database, Not in PHP

```php
// heavy: all rows loaded into memory, summed in PHP
$total = Article::all()->sum('word_count');

// light: the database does the summing
$total = Article::sum('word_count');

// per-author summary: top 5 authors by word count
Article::selectRaw('author_id, COUNT(*) AS total, SUM(word_count) AS total_words')
    ->groupBy('author_id')
    ->orderByDesc('total_words')
    ->limit(5)
    ->get();
```

> Like asking the bank for your balance, instead of requesting every transaction and adding them up yourself.

Use the Stored Total

```
// word_count is computed once, when the article is saved
$article->word_count = str_word_count($article->body);

// the report just sums the stored column
Article::whereBetween('published_at', [$from, $to])->sum('word_count');
```

- Derived values are computed on the server at creation time (Meeting 6)
- Reports do not recompute from raw data every time they open

Tools for Large Datasets

Technique	Use when	Note
`paginate()`	showing a list	one page per request
`chunkById()`	updating many rows	safe when rows change
`lazyById()`	reading many rows like a `foreach`	flat memory
`cursor()`	one long query	less safe if data changes
`select([...])`	only some columns needed	smaller models
`with()`	using relations in a loop	avoids N+1
index	filter or sort on a column	no full table scan

Index: A Table of Contents for the Table

```
Schema::table('articles', function (Blueprint $table) {
    $table->index('published_at');
});
```

```
EXPLAIN SELECT * FROM articles WHERE published_at BETWEEN '2026-08-01' AND '2026-08-31';
-- type: ALL    = full table scan (slow)
-- type: range  = uses the index (fast)
```

Without an index, the database reads the table row by row. With one, it jumps straight to the rows it needs.

> Recall Meeting 4: index strategy and `EXPLAIN`

Part 3

Memory Management

Why Memory Runs Out

- PHP has a `memory_limit` (commonly 128 MB)
- Each Eloquent row becomes a **model object**, much bigger than the raw data
- `get()` loads every row at once

> Jobsheet example: reading 52,500 transactions with `get()` uses about 96 MB, while `lazyById()` uses only about 38 MB. Add a bit more data and `get()` hits *Allowed memory size exhausted*.

`get` vs `chunk` vs `lazy` vs `cursor`

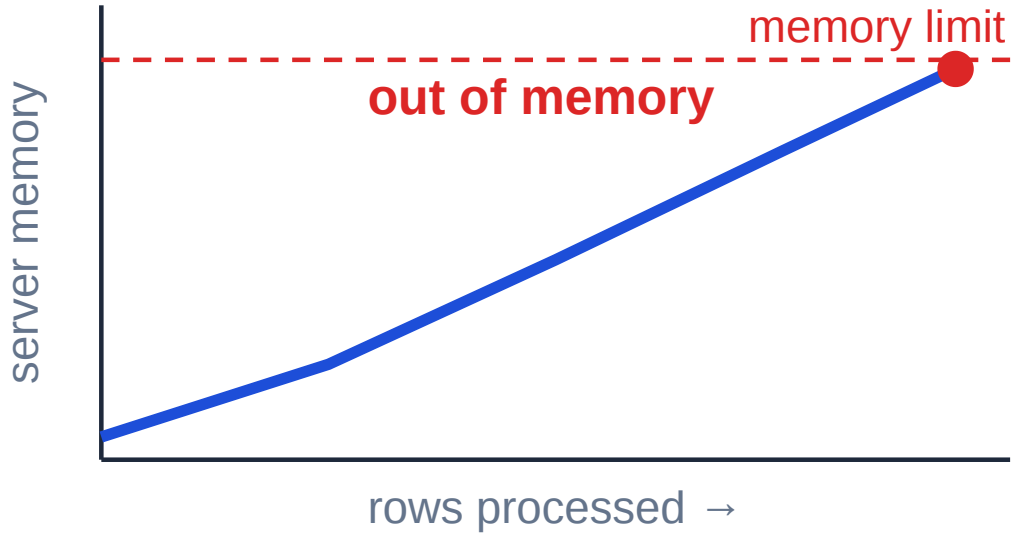
Method	Memory	Queries	Good for
`get()`	all rows	1	small data
`chunkById(n)`	n rows	many	bulk updates
`lazyById(n)`	n rows	many	export, plain `foreach`
`cursor()`	1 row	1	one-pass reads, data not changing

> Could be hundreds of thousands of rows? Do not `get()`. Read with `lazyById()`, update with `chunkById()`.

Bucket vs Tap

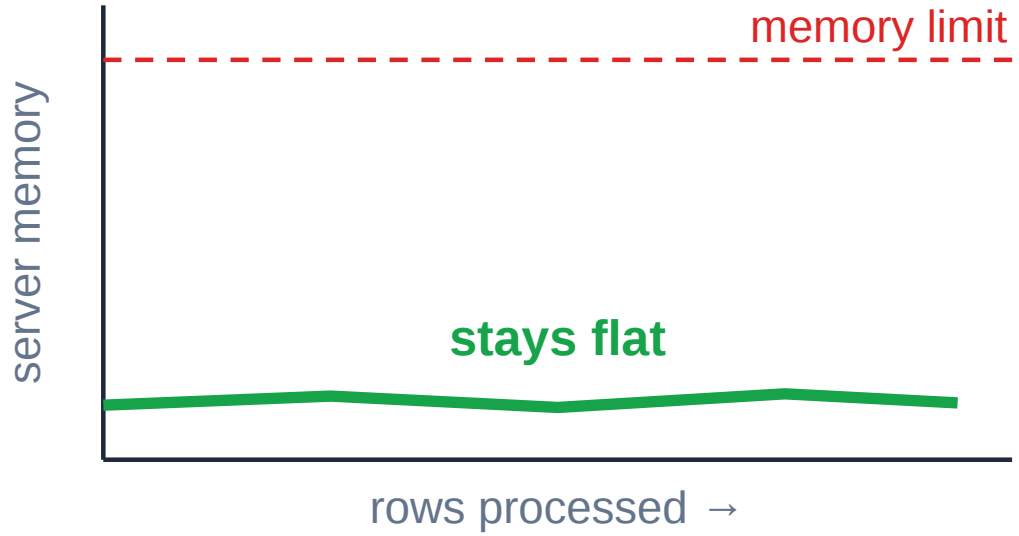
Bucket: collect first

get() then build one big string



Tap: flow row by row

lazyById() + streamDownload()



Export with `streamDownload()` and `lazyById()`

```php
$articles = Article::with('author')
    ->whereBetween('published_at', [$from, $to]);

return response()->streamDownload(function () use ($articles) {
    $out = fopen('php://output', 'w');
    fputcsv($out, ['Title', 'Author', 'Date']);
    foreach ($articles->lazyById() as $a) {
        fputcsv($out, [$a->title, $a->author->name, $a->published_at]);
    }
    fclose($out);
}, 'articles-report.csv');
```

`lazyById()` reads per chunk, `streamDownload()` sends per row. Memory stays flat.

> Laravel docs: `HTTP Responses → File Downloads` and `Eloquent → Chunking`

Don't Guess, Measure

```php
gc_collect_cycles();
memory_reset_peak_usage();      // start counting the peak from zero

foreach (Article::lazyById() as $a) {
    // ... work ...
}

printf("peak %.1f MB\n", memory_get_peak_usage() / 1048576);
```

- `memory_get_peak_usage()` : the highest memory use so far
- `memory_reset_peak_usage()` (PHP 8.2+): each measurement stands alone
- Compare two approaches on the same data, then write down the numbers

The `chunk()` Trap: Skipped Rows

```
// DANGEROUS: filters on 'draft', then changes 'draft' itself
Article::where('status', 'draft')->chunk(100, function ($articles) {
    foreach ($articles as $a) {
        $a->update(['status' => 'published']);
    }
});
```

Updated rows leave the filter, the list shifts, and the next chunk skips some rows.

> Fix: `chunkById(100, ...)`. It uses the last `id` as a marker, so it is safe even when rows change.

Part 4

Formats and Libraries: CSV, Excel, PDF

Choosing a Format: CSV, Excel, or PDF

	CSV	Excel (.xlsx)	PDF
Best for	raw data, between systems	analysis, financial reports	reading or printing
Memory	very light (stream)	medium	heavy (whole document)
Library	`fputcsv` (built into PHP)	Laravel Excel	DomPDF
Large data	yes	yes (`FromQuery`)	no, limit rows

CSV: Simple but Full of Traps

- **Delimiter**: Indonesian-locale Excel often uses a semicolon (`;`)
- **Quoting**: values with commas or newlines need quotes (`fputcsv` handles it)
- **Encoding**: UTF-8 from Excel often starts with an invisible character (**BOM**) that breaks the first header
- **No types**: every value is text, type validation is still your job

> Every row rejected on import? Check the header first: delimiter and BOM are the usual suspects.

Laravel Excel: Export

```php
class ArticlesExport implements FromQuery, WithHeadings, WithMapping, ShouldAutoSize
{
    public function query()
    {
        return Article::with('author')->orderBy('id');
    }

    public function headings(): array { return ['Title', 'Author', 'Date']; }

    public function map($article): array
    {
        return [$article->title, $article->author->name, $article->published_at];
    }
}

return Excel::download(new ArticlesExport, 'articles.xlsx');
```

You only describe the contents through small interfaces (`concerns`). The library handles the `.xlsx` format.

> docs.laravel-excel.com → Exports

Laravel Excel: Import

```php
class ArticlesImport implements ToModel, WithHeadingRow, WithValidation,
    WithChunkReading, WithBatchInserts
{
    public function model(array $row): Article
    {
        return new Article(['title' => $row['title'], 'author_id' => $row['author_id']]);
    }

    public function rules(): array
    {
        return ['title' => ['required'], 'author_id' => ['exists:authors,id']];
    }

    public function chunkSize(): int { return 1000; }   // read 1000 rows per turn
    public function batchSize(): int { return 500; }    // insert 500 rows per query
}

Excel::import(new ArticlesImport, $request->file('file'));
```

> By default, `WithValidation` stops at the first bad row. Add `SkipsOnFailure` to skip and collect bad rows.

PDF with DomPDF: HTML Becomes a Document

Blade view → **HTML + CSS** → **DomPDF** → **PDF file**

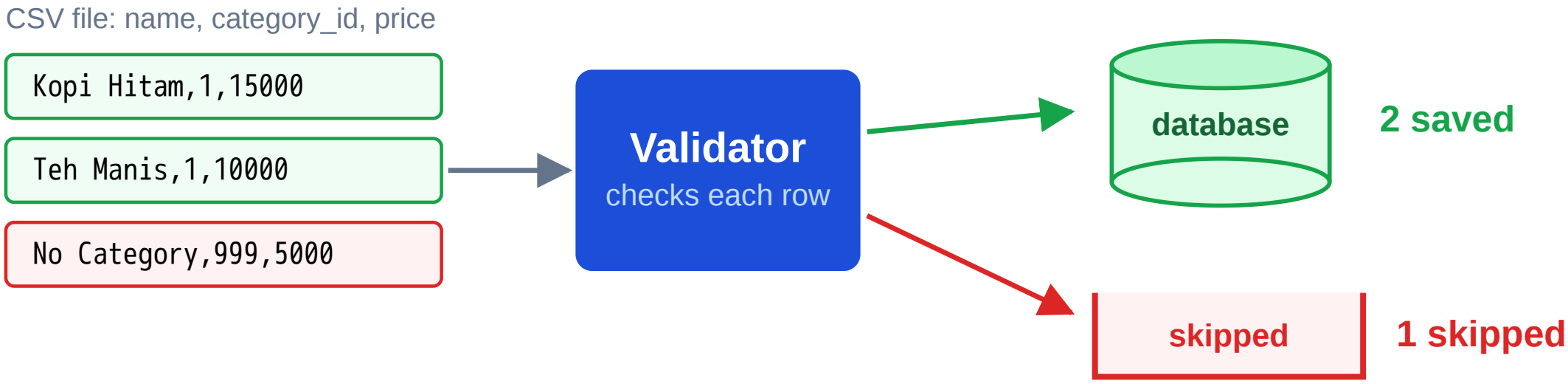
```
$pdf = Pdf::loadView('reports.articles', ['articles' => $articles])
    ->setPaper('a4', 'landscape');   // wide table: use landscape

return $pdf->download('articles.pdf');   // stream() to show in the browser
```

DomPDF supports only part of CSS and holds the whole document in memory. Use it for small summaries. For large data, pick CSV or Excel.

Import: One Bad Row Does Not Stop the Others

Like a cashier scanning groceries: a broken barcode is set aside, the rest keep scanning.



one bad row does not stop the others

Import: Validate Each Row, Skip the Bad Ones

```php
$rows = file($path, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
$rows = array_map('str_getcsv', $rows);
$header = array_shift($rows);

foreach ($rows as $row) {
    $data = array_combine($header, $row);

    $validator = Validator::make($data, [
        'slug' => ['required', 'string'],
        'title' => ['required', 'string'],
        'author_id' => ['required', 'exists:authors,id'],
    ]);

    if ($validator->fails()) {
        continue; // skip this row, go on to the next one
    }

    Article::updateOrCreate(['slug' => $data['slug']], $validator->validated());
}
```

What to Do with Bad Rows?

Strategy	How it works	Good for
Skip silently	skipped with no record	almost never: data vanishes quietly
Skip and log	skipped, row number and reason saved	general imports (recommended)
Abort all	one bad row cancels everything (`DB::transaction`)	financial or must-be-complete data

Part 5

Queue: The Journey of a Job

When Does Work Go to a Queue?

Directly (in the request)

- The user waits until it finishes
- Fits short work

Through a queue (job)

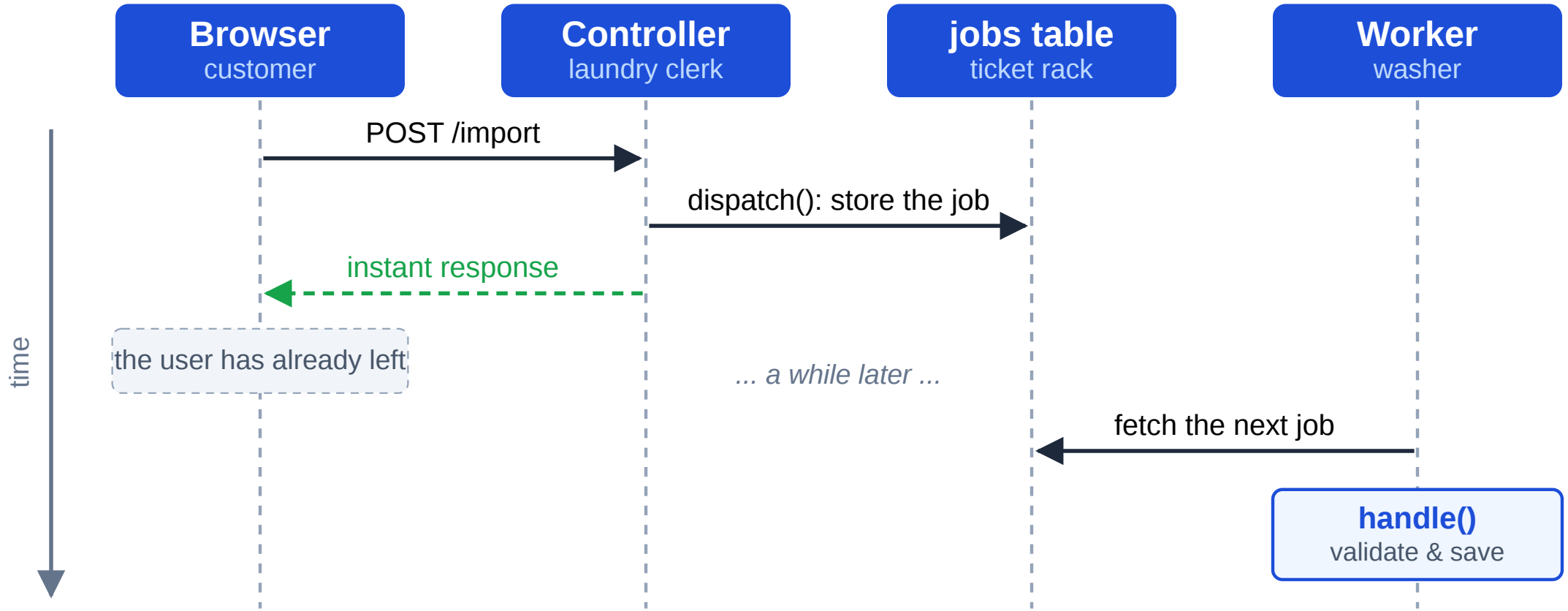
- Instant response, work in the background
- Fits heavy work

A task belongs in a queue when:

- its duration is unpredictable (depends on data size)
- the result is not needed immediately
- it calls a slow service (email, third-party API)
- it is safe to retry (idempotent)

> Laundry: one shirt you wait for, a whole sack you drop off with a ticket. An HTTP request has a time limit.

Queue = Laundry by the Kilo



One Example from Start to Finish

We follow one task: **importing articles from a CSV**.



> The next five steps use the same job: `ProcessArticleImport`.

Step 1: Create the Job Class

```
php artisan make:job ProcessArticleImport
```

```php
class ProcessArticleImport implements ShouldQueue
{
    use Queueable;

    public int $tries = 3;                // at most 3 attempts
    public array $backoff = [10, 60];     // wait 10 seconds, then 60 seconds
    public int $timeout = 60;             // cancel if it runs over 60 seconds

    public function __construct(public string $path) {}   // carry the path, not the file contents

    public function handle(): void
    {
        $rows = array_map('str_getcsv', file(Storage::path($this->path), FILE_IGNORE_NEW_LINES));
        $header = array_shift($rows);

        foreach ($rows as $row) {
            $data = array_combine($header, $row);
            Article::updateOrCreate(['slug' => $data['slug']], ['title' => $data['title']]);
        }
    }
}
```

Step 2: The Controller Sends the Job

```
public function store(Request $request)
{
    $request->validate(['file' => ['required', 'file', 'mimes:csv,txt']]);

    $path = $request->file('file')->store('imports');   // save the file first

    ProcessArticleImport::dispatch($path);               // hand the job to the queue

    return back()->with('success', 'The import is being processed in the background.');
}
```

- The controller only saves the file and sends the job, then responds right away
- The validate-and-save loop lives in `handle()` and has **not run yet**

Step 3: The Job Waits in the `jobs` Table

`dispatch()` only writes one row. No article has been imported yet.

id	queue	payload (short)	attempts	reserved_at
1	default	`{"displayName":"App\\Jobs\\ProcessArticleImport", ... "path":"imports/abc.csv"}`	0	NULL

- `payload`: the job class name and the data it carries (file path)
- `attempts = 0`: never tried
- `reserved_at = NULL`: no worker has picked it up

> Without a worker, these rows pile up. Tickets keep growing, but no laundry gets done.

Step 4: A Worker Executes the Job

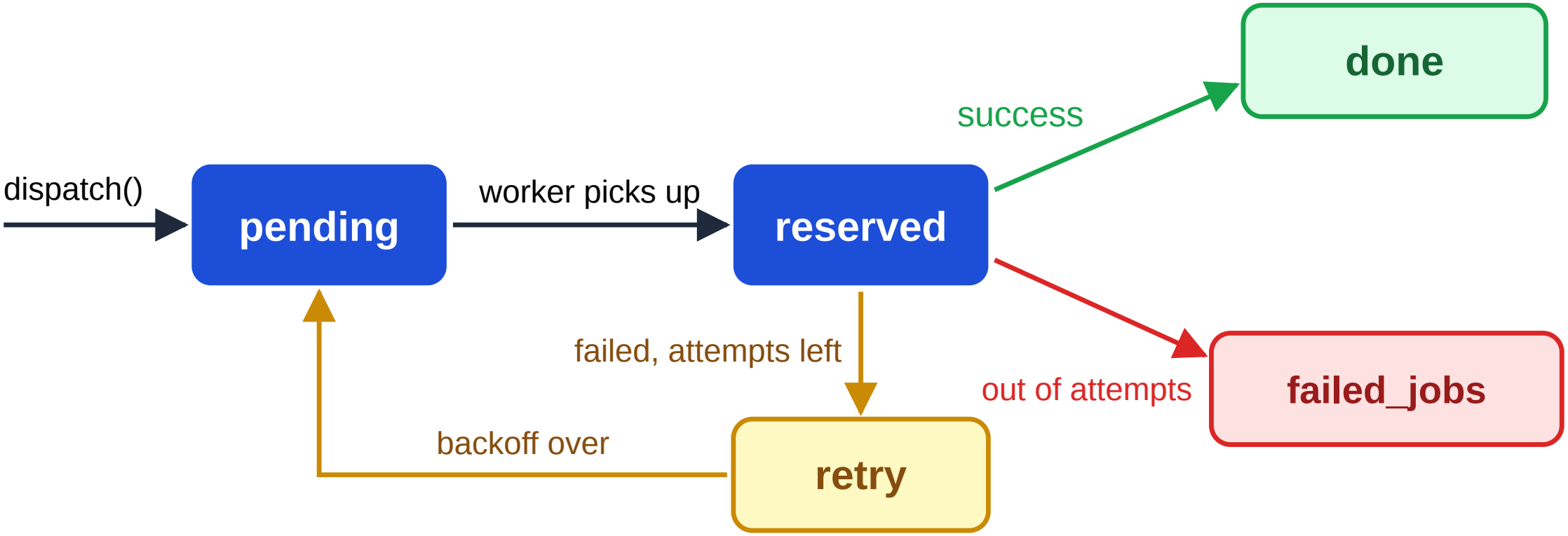
```
php artisan queue:work
```

```
  2026-10-21 10:00:00 App\Jobs\ProcessArticleImport ........ RUNNING
  2026-10-21 10:00:07 App\Jobs\ProcessArticleImport ... 7s DONE
```

1. The worker takes the row from `jobs`, sets `reserved_at`, and `attempts` becomes 1
2. It rebuilds the job object from `payload`, then calls `handle()`
3. On success the row is deleted from `jobs`. The articles are now in the database

> **Worker:** a long-running PHP process that takes jobs one by one and runs their `handle()`.

Step 5: When the Job Fails




```
$tries = 3, $backoff = [10, 60]
```

Step 5: Fail, Retry, then `failed_jobs`

```
  App\Jobs\ProcessArticleImport ........ RUNNING
  App\Jobs\ProcessArticleImport ... 1s FAIL     <- attempt 1, wait 10 seconds
  App\Jobs\ProcessArticleImport ........ RUNNING
  App\Jobs\ProcessArticleImport ... 1s FAIL     <- attempt 2, wait 60 seconds
  App\Jobs\ProcessArticleImport ........ RUNNING
  App\Jobs\ProcessArticleImport ... 1s FAIL     <- attempt 3 = $tries, goes to failed_jobs
```

```
php artisan queue:failed          # list failed jobs, with the exception
php artisan queue:retry all       # run them again after fixing the cause
php artisan queue:flush           # delete the failed job records
```

Summary of One Job's Flow



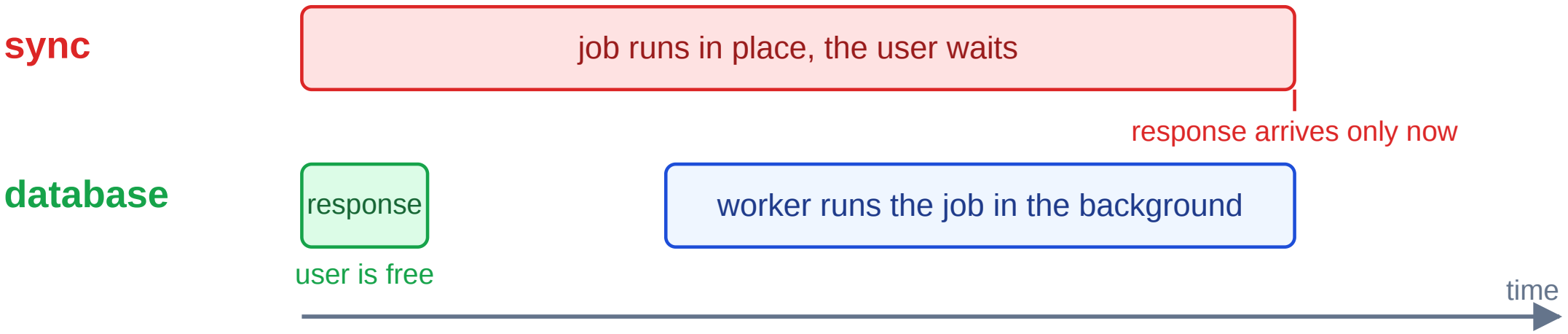
Success

- the `jobs` row is deleted
- the data is saved

Failure

- retry per `$tries` and `$backoff`
- exhausted: moves to `failed_jobs`

`sync` vs `database`: Who Waits?



> If `QUEUE_CONNECTION=sync` in `.env`, the job runs inside the request. There is no real queue.

Rules for Designing Jobs

Do

- Make it **idempotent** (`updateOrCreate`)
- Carry an ID or path, not big data
- Set `$tries`, `$backoff`, `$timeout`
- Use `->afterCommit()` if the job depends on data in a transaction

Avoid

- `request()`, session, or the logged-in user inside `handle()`: none exist in the background
- File contents in the job payload
- Relying on job order without designing for it
- Letting jobs fail with nobody watching

Keeping the User Informed

- **Status column** (`pending`, `processing`, `done`, `failed`) updated by the job and shown on a page
- **Notification** (email or in-app) when the job finishes or fails
- **Job batch**: `Bus::batch($jobs)->dispatch()` tracks many jobs, `$batch->progress()` gives a percentage

> Laravel docs: `Queues → Job Batching`

Part 6

Queues in Production

Setup: `.env` and the Queue Tables

```
# .env
QUEUE_CONNECTION=database
```

Driver	Storage	Good for
`sync`	none, the job runs in place	development, testing
`database`	the `jobs` table	small to medium projects
`redis`	Redis (in-memory)	high traffic

- A new Laravel project already has migrations for `jobs` and `failed_jobs`
- Older projects: `php artisan make:queue-table`, then `php artisan migrate`
- After changing `.env`: `php artisan config:clear` (or `config:cache` in production)

`retry_after` vs `--timeout`

```
// config/queue.php
'database' => [
    'driver' => 'database',
    'retry_after' => 90,    // after 90 seconds a job is seen as stuck and another worker takes it
],
```

Right

- `--timeout=60`
- `retry_after = 90`
- the job is stopped before it counts as stuck

Wrong

- `--timeout=120`
- `retry_after = 90`
- the job is still running, but a second worker takes it: **processed twice**

> Rule: `timeout` must be smaller than `retry_after`.

`queue:work` Options Worth Knowing

Option	Purpose
`--queue=high,default`	process `high` first, then `default`
`--tries=3`	max attempts per job
`--timeout=60`	seconds limit per job
`--sleep=3`	seconds to wait when the queue is empty
`--max-jobs=1000`	stop after n jobs (memory is released)
`--max-time=3600`	stop after n seconds
`--stop-when-empty`	stop when the queue is empty

> A worker that stops itself (`--max-time` , `--max-jobs`) is started again by Supervisor, so memory does not pile up.

Supervisor: Keeping Workers Alive

`queue:work` in a terminal dies when the terminal closes or the server restarts. Supervisor starts it again automatically.

```
; /etc/supervisor/conf.d/laravel-worker.conf
[program:laravel-worker]
process_name=%(program_name)s_%(process_num)02d
command=php /var/www/app/artisan queue:work --sleep=3 --tries=3 --max-time=3600
autostart=true
autorestart=true
stopasgroup=true
killasgroup=true
user=www-data
numprocs=2
redirect_stderr=true
stdout_logfile=/var/www/app/storage/logs/worker.log
stopwaitsecs=3600
```

Activating Supervisor

```
sudo apt-get install supervisor

sudo supervisorctl reread                       # read the new config file
sudo supervisorctl update                       # apply it
sudo supervisorctl start "laravel-worker:*"     # start all workers
```

- `numprocs=2` : two workers run at the same time
- `autorestart=true` : a worker dies, Supervisor starts it again
- `stopwaitsecs` : must be larger than the longest job, so jobs are not cut off

> Laravel docs: `Queues → Supervisor Configuration`

Deploy: Restart the Workers

Workers live long and keep the old code in memory. After a deploy, new code is not picked up until the workers restart.

```
git pull
composer install --no-dev --optimize-autoloader
php artisan migrate --force
php artisan config:cache
php artisan queue:restart     # workers stop after their current job finishes
```

> `queue:restart` does not cut off a running job. Workers stop cleanly, then Supervisor starts them with the new code.

Level Up: Redis and Horizon

- Start with `database`. Move to `redis` when traffic or job volume is high
- Set `QUEUE_CONNECTION=redis` and prepare a Redis server
- **Horizon**: dashboard and configuration for Redis queues (metrics, failed jobs, automatic worker count)
- Horizon is Redis only, and Supervisor keeps it alive too (`php artisan horizon`)

> Laravel docs: `Horizon` and `Queues → Supervisor Configuration`

Applying This to Simple POS

In the jobsheet you apply these concepts to Simple POS:

1. A sales report with a date filter, computed in the database
2. Report export to CSV (streaming), Excel, and PDF
3. Measuring memory for `get()` vs `lazyById()` on 50,000 rows
4. A product import from CSV that validates per row and is safe to repeat
5. Moving the import to a queued job with `$tries` and `backoff`, then tracing a failed job

> Full code: `github.com/se-polinema/simple-pos-ch08`

Summary (1/4): ETL, Batch, and Reports

- ETL: extract, transform, load. Import and export are two-way ETL
- Batching cuts round trips (`array_chunk` + `insert`, `chunkById`)
- Idempotent import: `updateOrCreate()` or `upsert()` (with a unique index)
- Reports: `whereBetween`, `startOfDay()` / `endOfDay()`, compute in the database, add an index

Summary (2/4): Memory and Formats

- `get()` loads every row. Read with `lazyById()`, update with `chunkById()`
- Measure with `memory_get_peak_usage()`, do not guess
- CSV for raw data, Excel for analysis, PDF for short documents
- Bad rows on import: skip and log, or abort for data that must be complete

Summary (3/4): The Journey of a Job

- `make:job`, then `dispatch()` only writes a row to the `jobs` table
- `queue:work` takes the row and runs `handle()`
- Failure: retry per `$tries` and `$backoff`, exhausted goes to `failed_jobs`, then `queue:retry`
- Idempotent jobs, carry an ID or path, never depend on the request or session

Summary (4/4): Queues in Production

- `QUEUE_CONNECTION=database` (or `redis`), the `jobs` table comes from a migration
- `timeout` must be smaller than `retry_after`
- Supervisor keeps `queue:work` alive (`numprocs`, `autorestart`, `stopwaitsecs`)
- Deploy: `queue:restart`. Large scale: Redis and Horizon

References & Discussion

Official Laravel docs (Queues, Horizon, File Storage, HTTP Responses, Eloquent Chunking), docs.laravel-excel.com, and laravel-dompdf

Full code: `github.com/se-polinema/simple-pos-ch08`

Next meeting: Designing and Building a REST API